

Exercise 2: Link State and Distance Vector Routing

Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm (เริ่มที่ router A)

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞ , —	∞ , —
1	B	2, A	5, A	5, B (2+3=5)	∞ , —
2	C	2, A	5, A	5, B (via C: 6 > 5)	6, C (5+1=6)
3	D	2, A	5, A	5, B	6, C (via D: 7 > 6)
4	E	2, A	5, A	5, B	6, C

เริ่มที่ A เพื่อนบ้าน B(ค่า 2) และ C(ค่า 5)

1: B มีค่าน้อยสุด (2) ยืนยัน B; ผ่าน B ถึง D = 2+3=5

2: C กับ D มีค่าเท่ากัน (5) ยืนยัน C; ผ่าน C ถึง E = 5+1=6

3: D มีค่า 5 ยืนยัน D; ผ่าน D ถึง E = 5+2=7 > 6 ไม่อัปเดต

4: E มีค่า 6 ยืนยัน E

Question 2 — Shortest-Path Tree (SPT) จาก A

Destination	Shortest Path	Total Cost
B	A -> B	2
C	A -> C	5
D	A -> B -> D	5
E	A -> C -> E	6

Question 3 — Forwarding Table ของ Router A

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A -> B
C	5	C	A -> C
D	5	B	A -> B -> D
E	6	C	A -> C -> E

Part 2 — Distance Vector Routing

Question 4 — Bellman-Ford Update สำหรับ Router A

สูตร Bellman-Ford: $D(A \rightarrow X) = \min \forall v \{ \text{cost}(A,v) + D(v \rightarrow X) \}$

cost: $\text{cost}(A,B)=2$, $\text{cost}(A,C)=5$

DV จาก B: $A=2, B=0, C=4, D=3, E=5$ | DV จาก C: $A=5, B=4, C=0, D=1, E=1$

Destination	Via B = 2 + D(B→X)	Via C = 5 + D(C→X)	Minimum Cost	Next Hop
B	2+0=2 ✓ vs 5+4=9	5+4=9	2	B
C	2+4=6	5+0=5 ✓ vs 2+4=6	5	C
D	2+3=5 ✓ vs 5+1=6	5+1=6	5	B
E	2+5=7	5+1=6 ✓ vs 2+5=7	6	C

ถึง B: ผ่าน B = 2+0=2 ✓ | ผ่าน C = 5+4=9 -> เลือก ผ่าน B (ค่า 2)

ถึง C: ผ่าน B = 2+4=6 | ผ่าน C = 5+0=5 ✓ -> เลือก ผ่าน C (ค่า 5)

ถึง D: ผ่าน B = 2+3=5 ✓ | ผ่าน C = 5+1=6 -> เลือก ผ่าน B (ค่า 5)

ถึง E: ผ่าน B = 2+5=7 | ผ่าน C = 5+1=6 ✓ -> เลือก ผ่าน C (ค่า 6)

Question 5 — Distance Vector Table ของ Router A หลังรับ DV

Destination	Next Hop	Distance	Full Path
B	B	2	A -> B
C	C	5	A -> C
D	B	5	A -> B -> D
E	C	6	A -> C -> E

Part 3 — Routing Convergence

Question 6 — ลิงก์ C-D ขัดข้อง: ผลต่อ LS และ DV

Link State Routing	Distance Vector Routing
พบความล้มเหลว: C และ D พบว่าหายไปผ่าน Hello/keepalive packet ที่ไม่ตอบสนอง ข้อมูลที่ flood: C และ D สร้าง LSA (Link State Advertisement) ใหม่ ประกาศว่าลิงก์ C-D หายไป แล้ว flood ทั่วเครือข่าย อัปเดต LSDB: ทุก router รับ LSA และอัปเดต Link-State Database ของตัวเอง ทำให้มี topology map เดียวกัน คำนวณเส้นทางใหม่: ทุก router รัน Dijkstra อีกครั้ง Convergence เร็วเพราะข้อมูล topology แพร่กระจายโดยตรง	C และ D ตั้งค่าระยะทางถึงกันเป็น ∞ และส่ง DV ที่อัปเดตให้เพื่อนบ้าน การแพร่กระจาย DV: เวกเตอร์อัปเดตไหลผ่านทีละ hop จนกว่าทุก router จะรับรู้ ซ้ำกว่า LS ข้อมูลผิดพลาด: router อื่นอาจยังคิดว่าถึง D ผ่าน C ได้อยู่ จนกว่า DV ที่อัปเดตจะมาถึง อาจเกิด routing loop ชั่วคราว Convergence ช้า: ต้องรอหลาย round ของการแลกเปลี่ยน DV โดยเฉพาะถ้าเกิด count-to-infinity

Part 4 — Count-to-Infinity

Question 7

1. ทำไมถึงเกิด routing loop?

เมื่อ B-C ขาด B ตั้งค่าระยะทางถึง C เป็น ∞ แต่ก่อนที่ส่ง DV ออกไป A ส่ง DV มาบอก B ว่า "A ถึง C ได้ค่า 2" B จึงคิดว่ายังมีเส้นทางไปถึง C ผ่าน A ได้ด้วยต้นทุน $2+1=3$ ซึ่งจริงๆ แล้วเส้นทางนั้นคือ $B \rightarrow A \rightarrow B \rightarrow C$ ที่วนซ้ำ เกิด **routing loop**

2. ค่าประมาณระยะทางถึง C เพิ่มขึ้นตามรอบ:

รอบ (Iteration)	D(A -> C)	D(B -> C)	หมายเหตุ
เริ่มต้น	2 (ผ่าน B)	1 (direct)	สถานะปกติ
1 (B-C ขาด)	2 (ยังไม่รู้)	3 (ผ่าน A: 1+2)	B อัปเดตจาก A
2	4 (ผ่าน B: 1+3)	3	A อัปเดตจาก B
3	4	5 (ผ่าน A: 1+4)	B อัปเดตจาก A
4	6 (ผ่าน B: 1+5)	5	วนซ้ำต่อเนื่อง
...	เพิ่มขึ้นเรื่อยๆ	เพิ่มขึ้นเรื่อยๆ	จนถึงค่า infinity

3. Count-to-Infinity คืออะไร?

Count-to-infinity คือปรากฏการณ์ใน Distance Vector routing ที่ router สองตัว (หรือมากกว่า) ต่างบอกกันว่ายังถึงปลายทางได้ ทำให้ค่าระยะทางเพิ่มขึ้นเรื่อยๆ โดยไม่รู้ว่เส้นทางที่อ้างอิงนั้นวนผ่านตัวเองจนถึงค่าสูงสุดที่กำหนด (infinity) กว่าจะหยุด

4. ทำไมเกิดขึ้นใน Distance Vector?

เพราะ DV router รู้แค่ถึงปลายทางได้ด้วยค่าเท่าไร แต่ไม่รู้ว่เส้นทางนั้นผ่านใครบ้าง (no full topology knowledge) ทำให้ไม่สามารถตรวจพบว่เส้นทางที่ตัวเองใช้วนลูบผ่านตัวเองหรือไม่

5. วิธีแก้ไข (Mitigation)

- Split Horizon: Router จะไม่ส่งข้อมูลเกี่ยวกับเส้นทางที่ได้มาจากเพื่อนบ้านกลับไปหาเพื่อนบ้านนั้น

- Split Horizon + Poison Reverse: แทนที่จะไม่ส่ง ให้ส่งค่า ∞ กลับไปแทน ทำให้เพื่อนบ้านรู้ว่าเส้นทางนี้ไม่ควรใช้
- กำหนดค่า infinity สูงสุด เช่น RIP ใช้ 16 hop = infinity
- เปลี่ยนมาใช้ Link State routing ที่ไม่มีปัญหา count-to-infinity

Part 5 — เปรียบเทียบ LS และ DV

Question 8 — ตารางเปรียบเทียบ Link State vs Distance Vector

Characteristic	Link State	Distance Vector
อัลกอริทึมพื้นฐาน	Dijkstra's Shortest Path Algorithm	Bellman-Ford Equation
ข้อมูลที่เก็บ	LSDB — แผนที่ topology เต็มรูปแบบของเครือข่าย	DV Table รู้แค่ค่าใช้จ่ายถึงเพื่อนบ้านและระยะทางที่เพื่อนบ้านบอกมา
ข้อมูลที่แลกเปลี่ยน	LSA (Link State Advertisement) — flood ทั่วเครือข่าย	Distance Vector ทั้งตาราง แจ้งเพื่อนบ้านโดยตรงเท่านั้น
Topology knowledge	Global view รู้ topology ทั้งเครือข่าย	Local view รู้แค่เพื่อนบ้านโดยตรง
Routing updates	Triggered + periodic refresh	Periodic updates หรือ triggered updates
Convergence	เร็วกว่า flood LSA ได้รวดเร็ว	ช้ากว่า ข้อมูลต้องผ่านทีละ hop
Routing loops	ไม่เกิด มี topology ครบถ้วน	อาจเกิดชั่วคราวระหว่างการ convergence
Count-to-infinity	ไม่เกิด	เกิดได้ แก้ด้วย Split Horizon, Poison Reverse
ตัวอย่าง Protocol	OSPF, IS-IS	RIP, EIGRP (hybrid), BGP (path vector)

Part 6 — Design Question

Question 9

คำตอบ: เลือก Link State Routing เพราะ

1. Convergence เร็ว

เครือข่าย 500 routers ต้องการ convergence ที่รวดเร็วเมื่อลิงก์หรือ router ขัดข้อง LS routing flood LSA ไปทั่วเครือข่ายทันที ทุก router สามารถคำนวณเส้นทางใหม่ได้เองด้วย Dijkstra ภายในเวลาสั้น ต่างจาก DV ที่ต้องรอ hop-by-hop และอาจใช้เวลาหลายนาที (slow convergence = downtime)

2. Topology Knowledge สมบูรณ์

LS router ทุกตัวมีแผนที่ topology เต็มรูปแบบ ทำให้คำนวณเส้นทางสั้นสุดได้แม่นยำ ไม่มี routing loop และไม่มี count-to-infinity สำหรับองค์กรที่ต้องการ efficient routing 500 routers นั้นสำคัญมาก เพราะ DV อาจเลือกเส้นทางที่ไม่เหมาะสมเพราะข้อมูลไม่ครบถ้วน

3. รองรับ Hierarchical Design ด้วย Areas (OSPF)

OSPF รองรับการแบ่ง network ออกเป็น Areas ซึ่งตรงกับความต้องการ hierarchical design ขององค์กร สามารถแบ่ง 500 routers ออกเป็นหลาย area โดยมี Area 0 (backbone) เป็น core ทำให้ลดขนาด LSDB ในแต่ละ area ลดปริมาณ LSA flooding และทำให้เครือข่ายขนาดใหญ่บริหารจัดการได้ง่ายขึ้น

Challenge — Algorithmic Analysis

Question 10 — วิเคราะห์อัลกอริทึม LS vs DV สำหรับ N routers, E links

Metric	Link State	Distance Vector
ข้อมูลที่เก็บ	LSDB ขนาด $O(N+E)$ ต่อ router ต้องเก็บ LSA ของทุก router ในเครือข่าย	DV Table ขนาด $O(N)$ ต่อ router เก็บแค่ระยะทางถึงแต่ละปลายทาง
Communication overhead	$O(N \times E)$ ทุก LSA ต้อง flood ทุก link แต่เกิดเฉพาะเมื่อมีการเปลี่ยนแปลง	$O(N^2)$ ต่อรอบ ทุก router ส่ง vector ขนาด N ให้เพื่อนบ้าน periodic
Computation	$O(N^2+E)$ หรือ $O((N+E)\log N)$ ด้วย priority queue (Dijkstra)	$O(N \times K)$ ต่อรอบ โดย K = จำนวนเพื่อนบ้าน (Bellman-Ford แต่ละ iteration เบา)
Convergence time	$O(\text{diameter})$ LSA flood ตาม diameter แล้วคำนวณเอง	$O(N)$ rounds worst case อาจหลาย iteration โดยเฉพาะ count-to-infinity
Scalability	ดีกว่าเมื่อใช้ hierarchical areas; อาจหนักเมื่อ N ใหญ่มากโดยไม่มี area	สื่อสารง่าย แต่ convergence ช้าเมื่อ N ใหญ่; count-to-infinity แยกเมื่อ path ยาวขึ้น

ทำไม intra-domain routing ขนาดใหญ่จึงนิยม Link State:

- ความแม่นยำ: LS มี global topology view ทำให้ Dijkstra หาเส้นทางสั้นสุดได้ถูกต้องเสมอ ไม่มี routing loop หรือ count-to-infinity
- Convergence เร็ว: ด้วยการ flood LSA แบบ reliable ทุก router อัปเดตพร้อมกัน ลด downtime ขณะ topology เปลี่ยน
- Hierarchical Scale: OSPF's area system แบ่งเครือข่ายใหญ่เป็น domain ย่อย ลด LSDB scope และ flooding overhead
- Predictability: ทุก router คำนวณจาก LSDB เดียวกัน ผลลัพธ์ consistent ทั่วเครือข่าย ง่ายต่อการ debug และ plan
- DV มีปัญหาที่ขยายตัว: ยิ่งเครือข่ายใหญ่ count-to-infinity ยิ่งใช้เวลานานกว่าจะ converge และ periodic update ทำให้ bandwidth ถูกใช้ต่อเนื่องแม้ไม่มีการเปลี่ยนแปลง